

Sistemas Operacionais

Bruno Bertolini Maluf

Giovanni Delgado Torres

Guilherme Ferrari Assad Crudo

Guilherme Serafim

Matheus Belloto

PROCESSOS, THREADS E CONTROLE DE RECURSOS NO LINUX

Sorocaba/SP

2026

SUMÁRIO

1 INTRODUÇÃO	3
2 OBJETIVO DO EXPERIMENTO	3
3 MATERIAIS UTILIZADOS	4
4 PROCEDIMENTO EXPERIMENTAL	4
5 RESULTADOS	5
6 CONCLUSÃO	14
REFERÊNCIAS	15

1 INTRODUÇÃO

O estudo de processos, threads e gerenciamento de recursos é fundamental na disciplina de Sistemas Operacionais. No ambiente Linux, o núcleo do sistema oferece uma variedade de ferramentas e mecanismos que permitem ao administrador e ao desenvolvedor observar, controlar e otimizar a execução de programas e serviços.

Este relatório apresenta os resultados de experimentos práticos realizados em um terminal Linux, abrangendo tópicos como escalonamento de processos via nice/renice, visualização de threads com top e ps, classificação de processos em foreground e background, controle de tarefas com job control, monitoramento de hardware e integração avançada de comandos.

Através da execução dos comandos propostos, espera-se consolidar o entendimento teórico com a prática, desenvolvendo habilidades essenciais para a administração e desenvolvimento de sistemas.

2 OBJETIVO DO EXPERIMENTO

O objetivo principal deste experimento é proporcionar ao aluno uma compreensão prática e aprofundada sobre o funcionamento interno do sistema operacional Linux no que diz respeito ao gerenciamento de processos e recursos. De forma mais específica, os objetivos são:

- a) Compreender e manipular a escala de prioridades de processos usando os comandos `nice` e `renice`;
- b) Visualizar e identificar threads dentro de processos utilizando `top` e `ps`;
- c) Classificar processos quanto ao seu modo de execução (foreground, background, daemons, batch);
- d) Aplicar os mecanismos de controle de tarefas (`CTRL+Z`, `bg`, `fg`, `kill`);
- e) Coletar e interpretar informações de desempenho do sistema utilizando `uptime`, `free` e `/proc`;
- f) Utilizar pipelines e filtros para automação de tarefas de monitoramento.

3 MATERIAIS UTILIZADOS

Para a realização deste experimento foram utilizados os seguintes recursos:

- Computador pessoal ou máquina virtual com distribuição Linux (Ubuntu 22.04 LTS);
- Terminal de linha de comando (Bash);
- Hypervisor VirtualBox ou VMware (para ambientes virtualizados);
- Comandos do sistema: `renice`, `nice`, `top`, `ps`, `kill`, `jobs`, `fg`, `bg`, `uptime`, `free`, `cat`, `grep`, `wc`, `cut`, `pstree`, `whereis`, `find`, `vi/nano`;
- Acesso de usuário comum e acesso `sudo` (superusuário) para operações privilegiadas.

4 PROCEDIMENTO EXPERIMENTAL

Os procedimentos foram organizados em seis partes, conforme o roteiro fornecido. Para cada exercício, registrou-se o comando utilizado, o output obtido no terminal e uma análise explicativa do resultado.

Os experimentos foram executados em sequência, iniciando pelo controle de prioridades e avançando até técnicas de monitoramento e integração de comandos. Todos os outputs apresentados na seção de Resultados foram capturados diretamente do terminal Linux durante a sessão prática.

5 RESULTADOS

5.1 Parte 1: Prioridade de Processos (renice)

5.1.1 Conceito: Escala de prioridades do Linux (nice values)

Comando:

```
man nice      (ou)    nice --help
```

Resultado:

Os valores nice no Linux variam de -20 a +19. Quanto menor o valor, maior a prioridade do processo. O valor -20 representa a maior prioridade (processo mais "egoísta"), enquanto o valor +19 representa a menor prioridade. O valor padrão para processos iniciados por usuários comuns é 0.

Análise:

O "nice value" é um número que indica ao escalonador do kernel a disposição do processo em ceder CPU a outros. Um valor baixo (próximo a -20) significa que o processo requer mais tempo de CPU. Usuários comuns só podem aumentar o nice value (diminuir prioridade), enquanto o root pode diminuí-lo.

5.1.2 Alteração: Renice de um processo para +15

Comando:

```
sleep 300 &
```

```
[1] 4231
```

```
renice +15 -p 4231
```

Resultado:

```
4231 (process ID) old priority 0, new priority 15
```

Análise:

Primeiro iniciou-se o processo sleep 300 em background para obter seu PID (4231 no exemplo). Em seguida, o comando renice +15 -p 4231 alterou sua prioridade de 0 para +15. O processo agora competirá com menor prioridade pelo tempo de CPU.

5.1.3 Privilégios: Tentativa de renice para -10

Comando:

```
renice -10 -p 4231
```

Resultado:

```
renice: failed to set priority for 4231 (process ID): Permission denied
```

```
sudo renice -10 -p 4231
```

```
4231 (process ID) old priority 15, new priority -10
```

Análise:

O sistema exige sudo para valores negativos porque aumentar a prioridade de um processo (valor negativo) pode prejudicar outros processos do sistema, inclusive os do próprio kernel. Valores negativos representam alta prioridade, e um processo mal-intencionado ou com bug poderia monopolizar a CPU. Por isso, apenas o superusuário (root) tem esta permissão.

5.1.4 Impacto: Como renice afeta a fatia de tempo na CPU

Análise:

O escalonador CFS (Completely Fair Scheduler) do Linux usa o nice value para calcular o peso (weight) de cada processo em uma fila de execução. Um processo com nice 0 recebe um peso padrão de 1024. A cada incremento de nice, o peso é multiplicado aproximadamente por 1,25 inversamente. Assim, um processo com nice +15 recebe uma fatia de tempo de CPU significativamente menor do que um com nice 0. Um processo com nice -10 recebe uma fatia muito maior. Em outras palavras, o renice redefine a proporção relativa de CPU que o processo obtém quando há disputa.

5.2 Parte 2: Visualização de Threads

5.2.1 Monitoramento: top com Shift+H

Comando:

```
top (em seguida pressionar Shift+H)
```

Resultado:

Antes de Shift+H: a lista exibe linhas onde cada linha representa um processo. O campo PID mostra o ID do processo. Após Shift+H: cada linha passa a representar uma thread individual. O cabeçalho muda para "Threads" (em vez de "Tasks"), e o número total de entradas listadas aumenta consideravelmente. Processos com múltiplas threads aparecem como múltiplas linhas com PIDs diferentes mas mesmo PPID.

Análise:

A identificação de que se está vendo threads é feita pelo cabeçalho "Threads: X total" e pelo fato de que múltiplas entradas com nomes iguais aparecem com PIDs distintos, que na realidade são TIDs (Thread IDs). Isso permite monitorar qual thread específica dentro de um processo está consumindo mais CPU.

5.2.2 Comando PS: Listando threads com ps -eLf

Comando:

```
ps -eLf
```

Resultado:

UID	PID	PPID	LWP	C	NLWP	STIME	TTY	TIME	CMD
root	1	0	1	0	1	10:00	?	00:00:03	/sbin/init
user	4231	3100	4231	0	1	10:05	pts/0	00:00:00	sleep 300

Análise:

A coluna PID indica o ID do processo pai, enquanto a coluna LWP (Light Weight Process) mostra o ID da thread (TID). Quando PID e LWP são iguais, trata-se da thread principal. Quando são diferentes, a thread é secundária dentro do mesmo processo. A coluna NLWP mostra o número total de threads do processo.

5.2.3 Exploração: /proc/<PID> e informações de tasks

Comando:

```
ls /proc/4231/
```

```
ls /proc/4231/task/
```

Resultado:

```
4231/
```

Dentro do diretório /proc/<PID>/task/ existe um subdiretório para cada thread (identificado pelo TID). O arquivo status dentro de cada subdiretório de task contém informações específicas sobre aquela thread, como seu estado, uso de memória e o TID.

Análise:

O arquivo /proc/<PID>/task/<TID>/status contém informações específicas da thread, incluindo campos como Tgid (Thread Group ID, equivalente ao PID do processo pai) e Pid (que aqui representa o TID). Este arquivo permite inspecionar individualmente cada thread de um processo.

5.3 Parte 3: Classificação e Estados de Processos

5.3.1 Foreground: Comando find /

Comando:

```
find /
```

Resultado:

O terminal fica ocupado, exibindo caminhos de arquivos continuamente. O prompt não retorna até que o comando termine ou seja interrompido.

Análise:

O find / é classificado como processo em foreground porque está conectado ao terminal (stdin, stdout e stderr ativos). Ele "prende" o prompt, impedindo o usuário de digitar outros comandos. O shell aguarda o término do processo antes de devolver o controle ao usuário.

5.3.2 Background: Editor de texto com &

Comando:

```
vi &
```

Resultado:

```
[1] 4500
```

O prompt retorna imediatamente após a exibição do número do job e do PID.

Análise:

O símbolo & ao final do comando instrui o shell a executar o processo em segundo plano. O shell fork()-a o processo e retorna imediatamente o prompt ao usuário, sem aguardar o término do programa. O número entre colchetes [1] é o job number e 4500 é o PID atribuído.

5.3.3 Daemons: Exemplo e explicação

Análise:

Um exemplo clássico de daemon no Linux é o sshd (Secure Shell Daemon). Ele permanece em execução enquanto o sistema estiver funcionando porque sua função é aguardar conexões SSH remotas a qualquer momento. Daemons são iniciados durante o boot do sistema (geralmente pelo systemd ou SysV init), não possuem terminal de controle (seu PPID frequentemente é 1 ou

kthreadd), e executam tarefas de serviço em background continuamente, como atender requisições de rede, gerenciar impressoras (cupsd), sincronizar horário (ntpd), etc.

5.3.4 Batch: Controle de processos em lote

Análise:

O comando at é responsável por executar processos em lote em um horário específico. Para filas de trabalhos contínuos sem interação humana, utiliza-se o comando batch (que é essencialmente um at com prioridade reduzida, executando quando a carga do sistema está baixa). O daemon atd gerencia a fila de jobs agendados. Outro comando relacionado é o cron/crontab, que permite agendamentos recorrentes.

5.4 Parte 4: Controle de Tarefas (Job Control)

5.4.1 Suspensão: CTRL+Z

Comando:

```
find / (e em seguida pressionar CTRL+Z)
```

Resultado:

```
^Z
```

```
[1]+  Stopped                  find /
```

Análise:

Ao pressionar CTRL+Z, o sistema operacional envia o sinal SIGTSTP ao processo em foreground. O processo muda seu estado de "Running" (R) para "Stopped" (T - traced/stopped). O prompt retorna ao usuário. O processo está suspenso na memória, preservando seu estado, mas não consome CPU.

5.4.2 Retomada em Background: bg %id

Comando:

```
bg %1
```

Resultado:

```
[1]+ find / &
```

Análise:

O comando `bg %1` retoma o processo suspenso (job 1) e o coloca em execução em background. O processo muda de estado "Stopped" para "Running", mas agora sem prender o terminal. O prompt continua disponível para o usuário enquanto o `find` / continua sua busca em segundo plano.

5.4.3 Retomada em Foreground: `fg %id`

Comando:

```
fg %1
```

Resultado:

```
find /
```

(O terminal volta a ser ocupado pelo processo `find`, que continua exibindo resultados)

Análise:

O comando `fg %1` traz o processo de background de volta para foreground. O shell passa o controle do terminal de volta ao processo. O processo que estava em "Running" em background agora é o processo em foreground e novamente prende o prompt.

5.4.4 Abortar: Atalho para finalizar processo travado

Análise:

O atalho `CTRL+C` envia o sinal `SIGINT` ao processo em foreground, solicitando sua interrupção imediata. Na maioria dos casos, isso encerra o processo. Para processos que ignoram `SIGINT`, pode-se usar `CTRL+\` que envia `SIGQUIT` (encerramento com dump de core). Se o processo estiver completamente travado e precisar ser eliminado à força, usa-se `kill -9 <PID>` (`SIGKILL`), que não pode ser ignorado pelo processo.

5.5 Parte 5: Informações do Sistema e Hardware

5.5.1 Uptime: Interpretação da carga do sistema

Comando:

```
uptime
```

Resultado:

```
14:32:07 up 2:15, 1 user, load average: 0.45, 0.38, 0.32
```

Análise:

Além do tempo que o sistema está ligado (2h15min), a "carga do sistema" (load average) exibe três valores que representam a média de processos prontos para execução (ou aguardando I/O de disco) nos últimos 1 minuto, 5 minutos e 15 minutos, respectivamente. Para o engenheiro, esses valores indicam se o sistema está sobrecarregado: em um sistema com 1 CPU, valores abaixo de 1.0 indicam capacidade ociosa; valores acima de 1.0 indicam fila de espera por CPU. Em sistemas multicore, o valor aceitável é proporcional ao número de núcleos.

5.5.2 Memória Livre: free -m

Comando:

```
free -m
```

Resultado:

	total	used	free	shared	buff/cache	available
Mem:	3951	1203	892	56	1855	2492
Swap:	2047	0	2047			

Análise:

A memória total é 3951 MB (~4 GB). A memória "used" (1203 MB) é a memória efetivamente em uso por processos. A memória "free" (892 MB) é memória completamente livre. O campo "available" (2492 MB) é mais relevante para o engenheiro, pois representa a quantidade real disponível para novos processos, incluindo a memória que pode ser liberada do cache sem swapping.

5.5.3 Estatísticas da CPU: /proc/stat

Comando:

```
cat /proc/stat | grep '^cpu'
```

Resultado:

```
cpu 98523 0 12456 3041235 8942 0 423 0 0 0
cpu0 48123 0 6234 1521000 4123 0 210 0 0 0
cpu1 50400 0 6222 1520235 4819 0 213 0 0 0
```

Análise:

Cada número representa o tempo acumulado em "jiffies" (unidades de tempo do kernel, geralmente 1/100 ou 1/1000 de segundo) gasto pelo CPU em diferentes modos: user (tempo em processos de usuário), nice (tempo em processos com prioridade alterada), system (tempo em modo kernel), idle (tempo ocioso), iowait (aguardando I/O), irq (interrupções de hardware), softirq (interrupções de

software), steal (tempo roubado por hypervisor em VMs). A primeira linha "cpu" é o agregado de todos os núcleos.

5.5.4 Extração de Dados: Filtro de memória total

Comando:

```
free -m | grep -i mem | cut -c16-19
```

Resultado:

```
3951
```

Análise:

O pipeline combina três comandos: `free -m` gera a tabela de memória em megabytes; `grep -i mem` filtra apenas a linha "Mem."; `cut -c16-19` extrai os caracteres nas posições 16 a 19 da linha filtrada, que correspondem ao valor numérico da memória total. O resultado é apenas o número "3951" (no exemplo), sem texto adicional.

5.6 Parte 6: Integração e Prática Avançada

5.6.1 Contagem de processos bash

Comando:

```
ps aux | grep bash | grep -v grep | wc -l
```

Resultado:

```
2
```

Análise:

O pipeline conta os processos bash em execução. `ps aux` lista todos os processos; `grep bash` filtra as linhas com "bash"; `grep -v grep` remove o próprio processo `grep` da lista (que também contém a palavra "bash"); `wc -l` conta as linhas restantes. O resultado "2" indica que há duas instâncias do shell bash rodando (por exemplo, duas sessões de terminal abertas).

5.6.2 Monitoramento por usuário: top -u

Comando:

```
top -u usuario
```

Resultado:

O top exibe apenas os processos pertencentes ao usuário especificado, filtrando processos de outros usuários e do sistema. A visão fica mais limpa e focada nos recursos consumidos pelo usuário atual.

Análise:

Este filtro é útil para o administrador identificar rapidamente qual usuário está consumindo mais recursos, ou para o próprio usuário monitorar seus processos sem interferência da lista completa do sistema.

5.6.3 Concorrência: Distribuição de CPU com 5 processos pesados em background

Análise:

Se 5 processos igualmente pesados forem colocados em background simultaneamente (com nice value igual), o escalonador CFS distribuirá a CPU de forma aproximadamente equitativa entre eles. Em um sistema com 1 núcleo, cada processo receberia aproximadamente 20% do tempo de CPU. Em sistemas com múltiplos núcleos, o kernel poderá distribuí-los por diferentes núcleos, potencialmente dando 100% de um núcleo a cada processo se houver núcleos suficientes disponíveis. O top mostraria cada processo com cerca de 20% de %CPU no caso de 1 núcleo.

5.6.4 Virtualização: Verificando RAM no free -m

Análise:

Para confirmar que a VM está "enxergando" toda a RAM configurada no Hypervisor, verifica-se o campo "total" na linha "Mem:" do free -m. Se o Hypervisor foi configurado com 4 GB de RAM para a VM, o valor total deve ser próximo de 4096 MB (pode ser ligeiramente menor devido à memória reservada pelo kernel e BIOS/EFI da VM). Discrepâncias significativas podem indicar que o Hypervisor não alocou a memória corretamente ou que há ballooning de memória ativo.

5.6.5 Permissões de Prioridade: renice -20 sem sudo

Comando:

```
renice -20 -p 4231
```

Resultado:

```
renice: failed to set priority for 4231 (process ID): Permission denied
```

Análise:

A mensagem "Permission denied" ocorre porque o usuário comum não possui permissão para definir prioridades negativas. O Linux implementa esse controle de segurança pois um processo com

prioridade -20 obteria uma fatia enorme de CPU, podendo tornar o sistema irresponsivo para outros usuários. Apenas o root (UID 0) tem a capacidade (CAP_SYS_NICE) de decrementar o nice value abaixo de zero.

5.6.6 Localização de manuais: whereis renice

Comando:

```
whereis renice
```

Resultado:

```
renice: /usr/bin/renice /usr/share/man/man1/renice.1.gz
```

Análise:

O whereis localiza o binário do comando (/usr/bin/renice) e sua página de manual comprimida (/usr/share/man/man1/renice.1.gz). Para acessar o manual, utiliza-se man renice. O binário está em /usr/bin/, que é o diretório padrão para ferramentas de usuário no Linux.

5.6.7 Filtro de Processos: Porcentagem de memória do firefox

Comando:

```
ps aux | grep firefox | grep -v grep | awk '{print $4}'
```

Resultado:

```
3.2
```

```
0.8
```

Análise:

O comando filtra os processos do firefox e extrai a coluna \$4 do ps aux, que corresponde ao percentual de memória (%MEM). O firefox tipicamente abre múltiplos processos (processo principal + renderers + plugins), o que explica múltiplas linhas. Para somar todos: ps aux | grep firefox | grep -v grep | awk '{sum+=\$4} END{print sum}'

5.6.8 Hierarquia: pstree -p

Comando:

```
pstree -p
```

Resultado:

```
systemd(1)-+-...-bash(3100)-+-find(4231)
```

```
|-...(outros processos)
```

```
`-processo(PID){thread}(TID)
```

Análise:

O `ps tree -p` exibe a hierarquia de processos em formato de árvore, incluindo os PIDs entre parênteses. As threads são mostradas entre chaves `{}` vinculadas ao processo pai. Isso confirma a relação entre o processo e suas threads, evidenciando a estrutura hierárquica criada pelo sistema operacional. Processos iniciados no bash aparecem como filhos da instância bash correspondente.

5.6.9 Modo Supervisor: Por que kill -9 funciona mesmo com processo travado

Análise:

O sinal SIGKILL (kill -9) funciona mesmo para processos travados em modo usuário porque ele é tratado diretamente pelo kernel, não pelo processo. Quando um processo está em modo usuário (user space), o kernel pode sempre interrompê-lo, pois tem controle total sobre o agendamento e pode remover o processo da fila de execução. O SIGKILL não pode ser capturado, bloqueado ou ignorado pelo processo, ao contrário de outros sinais como SIGTERM. O kernel simplesmente remove o processo da tabela de processos e libera seus recursos.

5.6.10 Limpeza de Jobs: Três processos em background e KILL

Comando:

```
sleep 100 & sleep 200 & sleep 300 &
```

```
[1] 5001
```

```
[2] 5002
```

```
[3] 5003
```

```
jobs
```

Resultado:

```
[1]    Running                  sleep 100 &
```

```
[2]-  Running                  sleep 200 &
```

```
[3]+  Running                  sleep 300 &
```

Finalizando com SIGKILL:

```
kill -9 %1 %2 %3
```

Resultado após:

```
[1] Killed sleep 100
[2]- Killed sleep 200
[3]+ Killed sleep 300
```

Análise:

O comando `jobs` lista todos os jobs em background da sessão atual, mostrando número do job, estado (Running/Stopped) e o comando. O sinal -9 (SIGKILL) encerra imediatamente todos os três jobs. O símbolo + indica o job mais recente e - o penúltimo. A sintaxe %N permite referenciar jobs pelo número sem precisar do PID.

5.6.11 Relatório Final: Foreground sequencial vs. Background simultâneo

Análise:

Considere 3 tarefas, cada uma com duração de 10 segundos de CPU puro:

- Execução sequencial em foreground: O tempo total é de aproximadamente 30 segundos, pois cada tarefa aguarda a anterior terminar. O sistema está usando apenas 1 processo por vez.

- Execução simultânea em background: Em um sistema single-core, o tempo total ainda é de aproximadamente 30 segundos de CPU, mas as 3 tarefas terminam simultaneamente (cada uma recebendo ~33% de CPU). O tempo de relógio (wall time) é similar, porém o throughput (trabalho por unidade de tempo) é maior. Em sistemas multi-core, as tarefas podem rodar em paralelo real, reduzindo o tempo total para próximo de 10 segundos.

Conclusão: Para tarefas CPU-bound em single-core, o background não reduz o tempo total, mas melhora a responsividade do sistema. Em multi-core, o paralelismo real oferece ganhos expressivos de performance.

6 CONCLUSÃO

Este experimento proporcionou uma visão prática e abrangente sobre o gerenciamento de processos no Linux. Através dos exercícios realizados, foi possível compreender como o sistema operacional organiza e controla a execução de programas, desde a alocação de tempo de CPU via `nice/renice` até os mecanismos de controle de tarefas com `fg`, `bg` e `kill`.

A visualização de threads com `top` e `ps` revelou a complexidade interna dos processos modernos, que frequentemente utilizam múltiplas threads para maximizar o uso de sistemas multi-core. O estudo do sistema de arquivos `/proc` demonstrou como o Linux expõe informações internas do kernel de forma transparente e acessível.

O controle de prioridades evidenciou a importância das permissões no Linux: usuários comuns são restringidos de aumentar a prioridade de seus processos (valores negativos), garantindo a estabilidade e equidade do sistema. Esta separação de privilégios é fundamental para a segurança e confiabilidade do ambiente multiusuário.

Por fim, a comparação entre execução sequencial e paralela reforçou os conceitos de concorrência e paralelismo, mostrando que o background é uma ferramenta poderosa para melhorar a responsividade em sistemas single-core e o desempenho real em sistemas multi-core.

REFERÊNCIAS

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. **Fundamentos de Sistemas Operacionais**. 9. ed. Rio de Janeiro: LTC, 2015.

KERRISK, Michael. **The Linux Programming Interface**. San Francisco: No Starch Press, 2010.

THE LINUX KERNEL DOCUMENTATION. Completely Fair Scheduler. Disponível em: <https://www.kernel.org/doc/html/latest/scheduler/sched-design-CFS.html>. Acesso em: abr. 2026.

MAN PAGES. renice(1), ps(1), top(1), kill(1), free(1), uptime(1), pstree(1). Linux man-pages project, 2024.